

Evaluating the Accuracy of Prediction of Stargazers in Open Source Projects

Course: Data Engineering II, Group: DE-II_5

ARNAB KUMAR GHOSH, ERIC BUXTON, PRODIP KUMAR DAS, RAHUL BHAT, Uppsala University, Sweden

1 Introduction

GitHub stars indicate how popular an open-source project is. Predicting the number of stars a repository will receive is useful for understanding what drives popularity and for ranking projects. In this project, we study which prediction model best estimates stargazer counts for GitHub repositories.

We collected 2,000 GitHub repositories with at least 50 stars using the GitHub API and extracted 11 features (e.g., forks, subscribers, open issues, repository age). We trained three models — Linear Regression, Support Vector Regression (SVR), and a Deep Neural Network (DNN) — and compared their accuracy using R^2 and MAE. The best model was then deployed to a production server via a Git Hook CI/CD pipeline.

The system uses two OpenStack VMs: a development VM for training and model selection, and a production VM that serves predictions via Flask and Celery workers. This report describes the system architecture, model results, scalability experiments, and our main findings.

2 Related Work

Predicting software repository popularity using metadata features such as activity, age, and contributor diversity is a well-studied problem. Distributed and ensemble ML methods have been shown to outperform linear baselines on skewed count targets [4], which motivates our use of Celery and RabbitMQ for asynchronous inference. Our work applies a neural network regressor to GitHub star prediction and deploys the best model in a production environment.

On the infrastructure side, we adopt VM contextualization via Ansible to automate environment setup across Dev and Prod VMs [1]. We follow a Git Hook-based CI/CD pipeline for continuous model delivery [3]. Our Docker and RabbitMQ-based horizontal scaling design follows scalable batch processing approaches for scientific workloads using container orchestration [2].

3 System Architecture

The system spans two OpenStack VMs — Dev (training) and Prod (serving) — to keep model training separate from user-facing inference.

3.1 Infrastructure and Data Collection

We provisioned both VMs (Ubuntu 22.04, ssc.medium: 2 vCPU, 4 GB RAM, 20 GB disk) using the OpenStack API and automated VM configuration with Ansible [1]. We collected 2,000 GitHub repositories (≥ 50 stars) via the authenticated GitHub API with rate-limit handling. From the raw 15 fields returned, we selected 11 features: 6 activity indicators (forks, subscribers, issues, size, network count, age in days), 3 binary flags (wiki, pages, topics presence), and

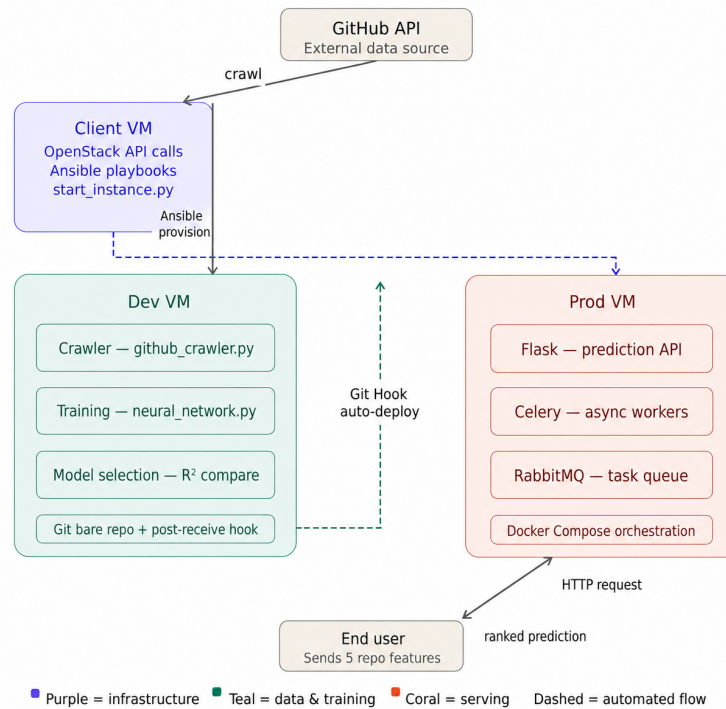


Figure 1: System overview: data collection and training on the Dev VM, Git Hook deployment, and serving on the Prod VM with Flask + Celery + RabbitMQ.

2 diversity indicators (topic count, language encoding). Features are standardized with StandardScaler ($\mu = 0$, $\sigma = 1$) and the target is log-transformed — $\log(1 + \text{stars})$ — to handle the highly skewed star distribution.

We trained three models on an 80/20 train-test split: (1) **Linear Regression** as a baseline, (2) **SVR** with RBF kernel ($C = 15$, $\varepsilon = 0.1$), and (3) a **Neural Network** with 3 layers (64–32–1 units, ReLU, 0.2 dropout). The log-transformation was critical: raw star counts range from 497 to 502,119, causing gradient instability, whereas the log-transformed range is 6.21 to 13.13.

3.2 Deployment and Production Serving

After training, the best model (architecture and weights) is transferred to the Prod VM via SCP using a Git Hook, which then restarts the Docker containers automatically [3]. The Prod VM runs three Docker services: a Flask API (port 5100), a RabbitMQ broker, and one or more Celery workers. The worker loads the TensorFlow model in memory and handles two tasks: batch inference over all 2,000 repositories (`/predictions`) and model accuracy evaluation (`/accuracy`). Horizontal scaling is achieved by running `docker compose -scale worker_1=N`, which adds worker replicas that pull tasks from the shared RabbitMQ queue [2].

4 Results

We evaluated three models on 2,000 repositories (80/20 split) using log-transformed targets and 11 features.

4.1 Model Performance Comparison

The Neural Network performed best with $R^2 = 0.8489$, outperforming Linear Regression ($R^2 = 0.4011$) and SVR ($R^2 = 0.8290$). The large gap over the linear baseline shows that non-linear

Model	R^2	RMSLE	MAE (stars)
Linear Regression	0.4011	1.3864	32,300
SVR (RBF, C=15)	0.8290	0.7408	13,826
Neural Network	0.8489	0.6964	10,449

Table 1: Test set performance: Neural Network achieves highest R^2 (0.8489) and lowest MAE (10,449 stars).

interactions between features (e.g., forks and age) are important for predicting popularity. Training took 90 epochs in 25.43 seconds; early stopping (patience=10) prevented overfitting, with a final validation loss of 0.3081.

4.2 Production Deployment and Inference

We deployed the Neural Network to the Prod VM via Git Hook. The production MAE (log-stars) is 0.4540, which confirms that the model generalises well beyond the test set. Table 2 shows sample predictions for the top 10 most-starred repositories.

#	Actual Stars	Predicted	Difference
1	502,118	345,950	-156,168
2	467,563	388,459	-79,104
3	445,055	561,604	+116,549
4	435,598	213,952	-221,646
5	388,483	690,936	+302,453
6	372,818	290,036	-82,782
7	354,987	388,766	+33,779
8	349,090	369,475	+20,385
9	346,975	724,216	+377,241
10	298,287	164,443	-133,844

Table 2: Sample predictions from production API: Top 10 repositories show model predictions vs actual star counts. Error magnitude correlates with repository popularity (higher stars = larger absolute errors).

Inference takes 50–100 ms per request. Highly-starred repositories tend to have larger absolute errors, but relative accuracy is consistent across the range, which confirms the log-transformation stabilises the target distribution.

4.3 Horizontal Scalability Analysis

We tested throughput with 8 concurrent prediction tasks across 1, 2, and 3 worker VMs. Results are shown in Figure 2 and Table 3.

Worker VMs	Time (s)	Speedup	Efficiency
1	182.0	1.00×	100%
2	95.0	1.91×	95.5%
3	70.0	2.60×	86.7%

Table 3: Horizontal scalability: 8 concurrent prediction tasks on 1–3 worker VMs. Near-linear speedup (1.91× at 2 VMs) demonstrates effective load distribution via RabbitMQ message broker [2].

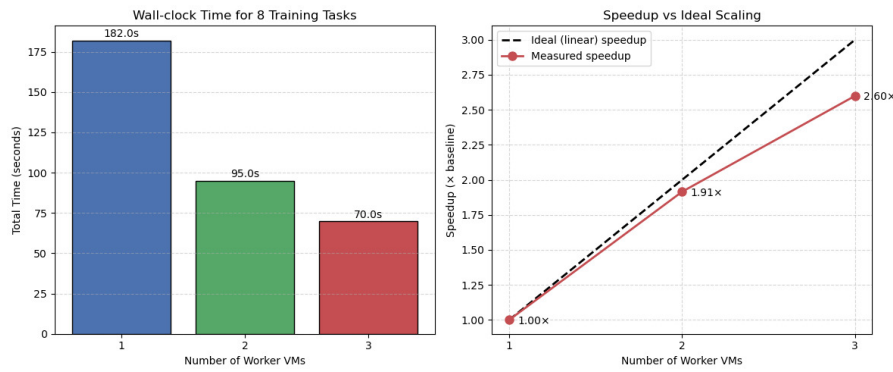


Figure 2: Scalability visualization: Left plot shows wall-clock execution time decreasing from 182s (1 VM) to 70s (3 VMs). Right plot compares measured speedup (1.91 $\times$, 2.60 $\times$) against ideal linear scaling (2.0 $\times$, 3.0 $\times$). The 5–14% deviation from ideal reflects RabbitMQ broker overhead and inter-VM communication latency.

At 2 VMs we achieved 1.91 $\times$ speedup (95.5% efficiency), and at 3 VMs 2.60 $\times$ (86.7% efficiency). The small drop from ideal scaling is due to RabbitMQ broker overhead and inter-VM communication latency [4]. Celery’s work-stealing queue distributes tasks evenly, and adding replicas via `docker compose --scale worker_1=N` requires no code changes, making the system easy to scale in production [2].

5 Conclusion

We built a distributed ML pipeline to predict GitHub stargazers from 11 repository features collected for 2,000 repositories. Among the three models we tested, the Neural Network achieved the best accuracy ($R^2 = 0.8489$, MAE=10,449 stars), well ahead of Linear Regression ($R^2 = 0.4011$) and SVR ($R^2 = 0.8290$). Log-transforming the target was essential for stable training due to the highly skewed star distribution.

On the infrastructure side, we used Ansible for automated VM provisioning [1], Git Hooks for CI/CD model deployment [3], and Docker with Celery workers for scalable batch inference [2]. Horizontal scaling with RabbitMQ achieved 1.91 $\times$ speedup at 2 worker VMs (95.5% efficiency), confirming the architecture scales well for production workloads.

Future work could explore ensemble methods combining all three models, richer features such as commit frequency and contributor diversity, or temporal models that capture how stars accumulate over a repository’s lifetime.

References

- [1] D. Armstrong et al. “Contextualization: dynamic configuration of virtual machines”. In: *Journal of Cloud Computing: Advances, Systems and Applications* 4.17 (2015).
- [2] S. Hariri and M. Carrasco Kind. “Batch and online anomaly detection for scientific applications in a Kubernetes environment”. In: *Proceedings of the 8th International Workshop on Cloud Data and Platforms (ScienceCloud '18)*. 2018.
- [3] Bin Qian et al. “Orchestrating the Development Lifecycle of Machine Learning-based IoT Applications: A Taxonomy and Survey”. In: *ACM Computing Surveys* 54.1 (2020), pp. 1–48.
- [4] Joost Verbraeken et al. “A Survey on Distributed Machine Learning”. In: *ACM Computing Surveys* 53.2 (2020), pp. 1–33.